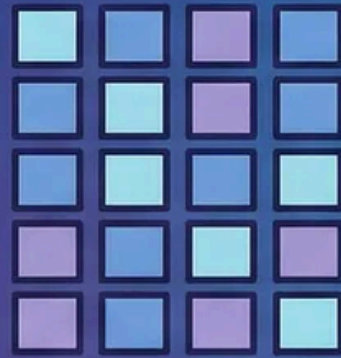


COMPUTER SCIENCE COURSE

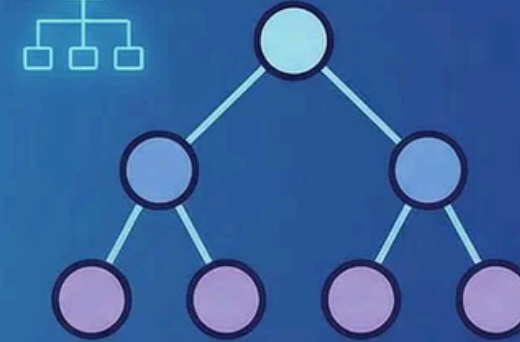
UNIVERSITY ACADEMIC SERIES



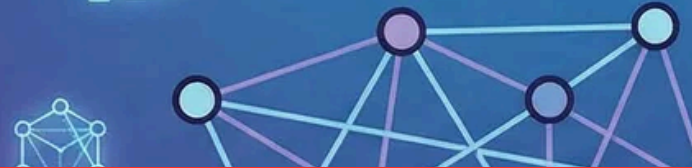
ARRAYS



LINKED LISTS



TREES



DATA STRUCTURES: BUILDING BLOCKS OF EFFICIENT COMPUTING

A Comprehensive Guide to Arrays,
Lists, Stacks, Queues, Trees, and
Graphs

COMPE 496
MURAT BALCI

WHY DATA STRUCTURES MATTER

Efficient data organization reduces algorithm complexity from $O(n^2)$ to $O(\log n)$

■ Performance Impact

Right structure = 1000x speed improvement.
Searching 1M items: Linear vs Binary Search.

■ Memory Efficiency

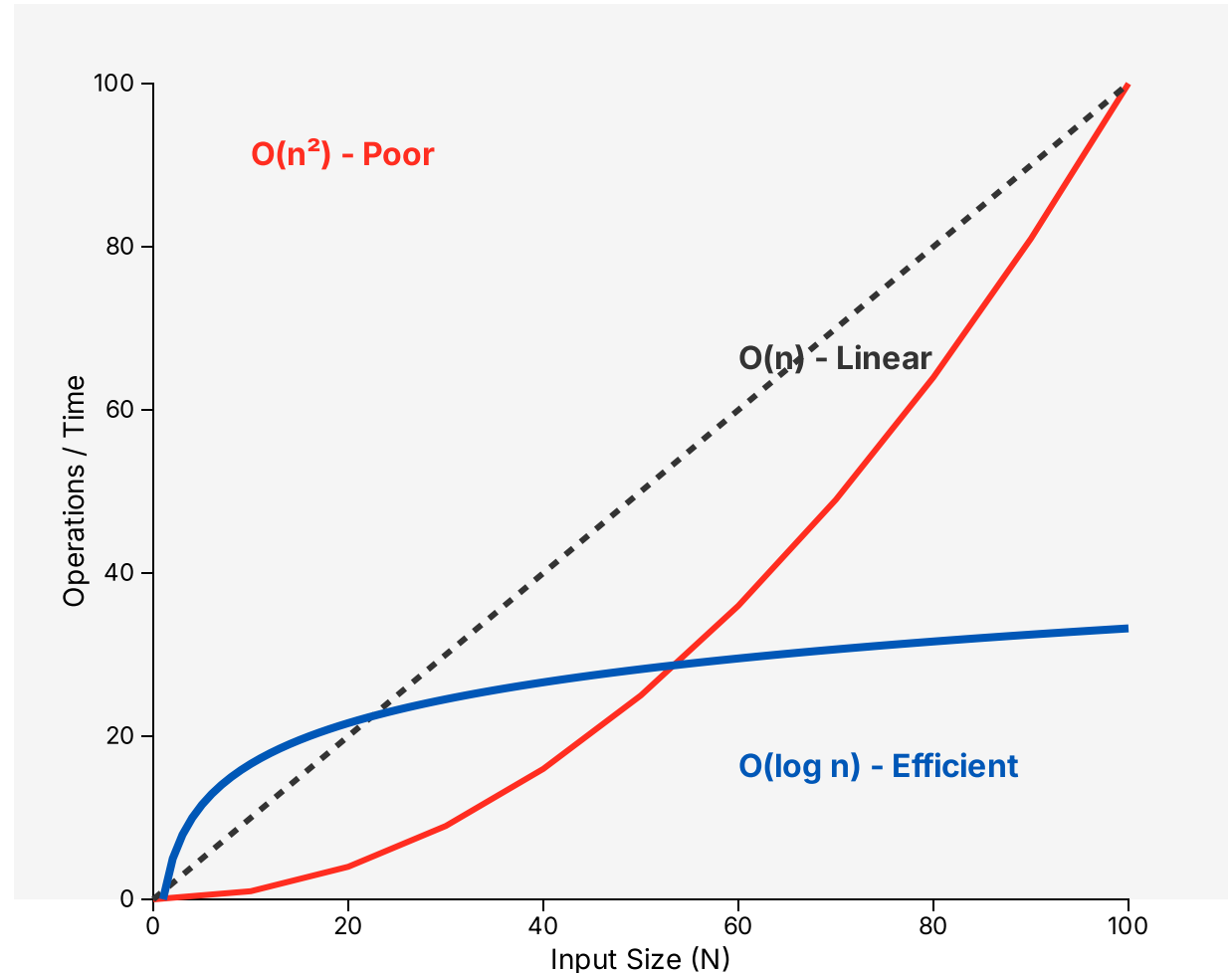
Proper organization saves storage space and optimizes cache usage.

■ Code Maintainability

Clear structure improves readability and reduces bugs in complex systems.

■ Industry Relevance

Core requirement for technical interviews and building scalable software.



ARRAYS: CONTIGUOUS MEMORY

Arrays provide $O(1)$ random access through contiguous memory allocation.

- **Fixed Size:** Determined at declaration time.
- **Direct Access:** `arr[i]` is $O(1)$ via index calculation.
- **Cache Friendly:** Spatial locality improves performance.
- **Memory Layout:** Elements stored sequentially.

```
int arr[5] = {10, 20, 30, 40, 50};  
cout << arr[2]; // Output: 30
```

0	1	2	3	4
10	20	30	40	50
0x1000	0x1004	0x1008	0x100C	0x1010

ADDRESS CALCULATION

$$\text{Addr} = \text{Base} + (\text{Index} \times 4)$$

$$0x1008 = 0x1000 + (2 \times 4)$$

ARRAY OPERATIONS

Insertion and deletion require shifting elements, resulting in $O(n)$ complexity.

OPERATION	COMPLEXITY	NOTE
Access	$O(1)$	Direct index
Search	$O(n)$	Linear scan
Insertion	$O(n)$	Requires shifting
Deletion	$O(n)$	Requires shifting

While arrays offer instant access, they are rigid. Inserting an element in the middle forces all subsequent elements to move, creating a performance bottleneck for dynamic data.

STEP 1: SHIFT ELEMENTS RIGHT



STEP 2: INSERT NEW VALUE (25)



```
// Insert 25 at index 2
for(int i = 4; i >= 2; i--) {
    arr[i+1] = arr[i]; // Shift right
}
arr[2] = 25;
```

DYNAMIC ARRAYS

Dynamic arrays automatically resize by allocating new memory and copying elements.

- **Automatic Resizing:** Manages memory internally.
- **Growth Strategy:** Typically doubles capacity when full.
- **Amortized $O(1)$:** Resizing is rare; average insertion is constant time.
- **Contiguous Memory:** Maintains cache efficiency.

```
std::vector<int> vec = {10, 20, 30};  
// Capacity: 3, Size: 3  
  
vec.push_back(40); // Triggers resize  
// Capacity: 6, Size: 4
```

STATE 1: FULL CAPACITY CAP: 3



↓ Resize & Copy

STATE 2: NEW ALLOCATION CAP: 6

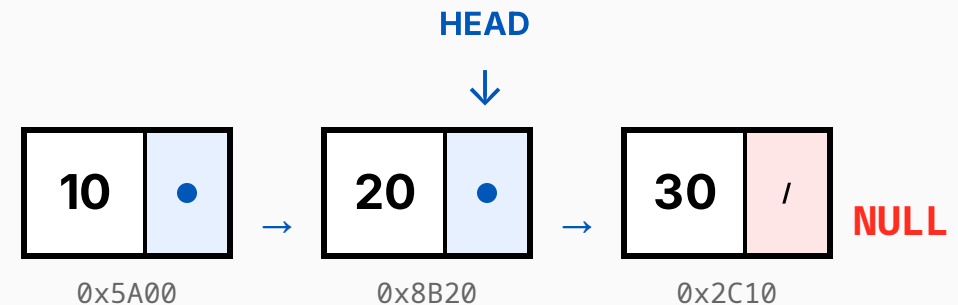


LINKED LISTS: DYNAMIC NODES

Linked lists enable $O(1)$ insertion/deletion by connecting nodes through pointers.

- **Non-Contiguous:** Nodes scattered in memory.
- **Dynamic Size:** Grow/shrink without reallocation.
- **Insertion/Deletion:** $O(1)$ at known position.
- **Random Access:** $O(n)$ traversal required.

```
struct Node {  
    int data;  
    Node* next;  
};  
  
Node* head = new Node{10, nullptr};
```



Note: Addresses are random (non-sequential)

LINKED LIST OPERATIONS

Pointer manipulation enables efficient insertion without data movement.

Insert at Head

Update head pointer only

$O(1)$

Insert After Node

Update 2 pointers (no shifting)

$O(1)$

Delete Node

Bypass node (given prev ptr)

$O(1)$

Insert at Tail

Traverse to end (unless tail ptr)

$O(n)$

Search

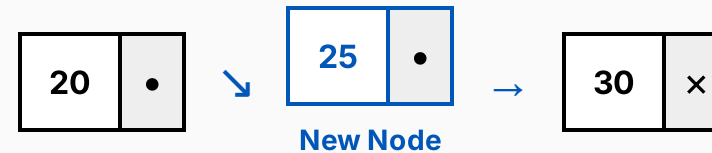
Linear traversal required

$O(n)$

```
// Insert 25 after node 20
```

```
Node* newNode = new Node{25, current->next};  
current->next = newNode; //  $O(1)$  insertion
```

VISUALIZING INSERTION



DOUBLY LINKED LISTS

Doubly linked lists add backward pointers for $O(1)$ reverse traversal and deletion.

- **Bidirectional Navigation:** Each node contains prev and next pointers, allowing traversal in both directions.
- **$O(1)$ Deletion:** Can delete a node given only a pointer to it, as we can access the previous node directly.
- **Memory Overhead:** Requires extra memory for the second pointer (8-16 bytes per node).
- **Implementation:** C++ STL `std::list` uses this structure.

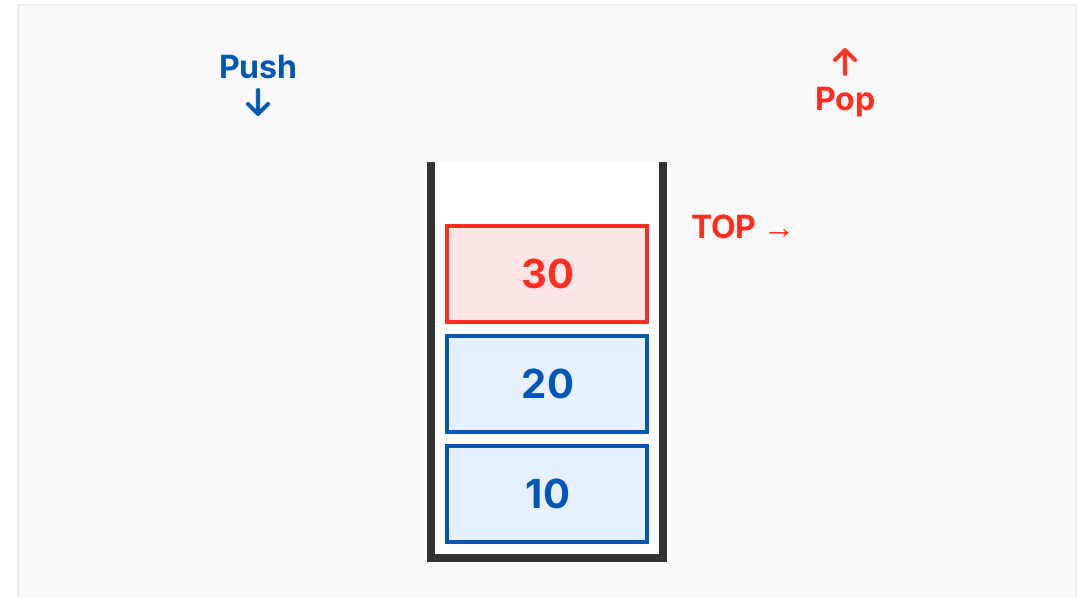


```
struct DNode { int data; DNode* prev; DNode* next;
}; // Delete a node (given pointer to it) void
deleteNode(DNode* node) { if(node->prev) node->prev-
>next = node->next; if(node->next) node->next->prev
= node->prev; delete node; //  $O(1)$  operation }
```


STACKS: LIFO PRINCIPLE

Stacks restrict access to the most recently added element, enabling $O(1)$ push and pop.

- **LIFO (Last-In, First-Out):** The last element added is the first one removed.
- **Core Operations:**
 - - **Push:** Add to top
 - - **Pop:** Remove from top
 - - **Peek:** View top element
- **Efficiency:** All operations are $O(1)$ constant time.
- **Applications:** Function calls, undo mechanisms, expression parsing.



```
std::stack<int> s; s.push(10); // Stack: [10]
s.push(20); // Stack: [10, 20] s.push(30); //
Stack: [10, 20, 30] cout << s.top(); // 30 (peek)
s.pop(); // Stack: [10, 20]
```

STACK IMPLEMENTATION

Array-based stacks offer cache efficiency while linked stacks provide unlimited growth.

FEATURE	ARRAY STACK	LINKED STACK
Size	Fixed (Static)	Dynamic (Growable)
Memory	Efficient (No ptrs)	Overhead (Per node)
Performance	Cache Friendly	Pointer Chasing

KEY APPLICATIONS

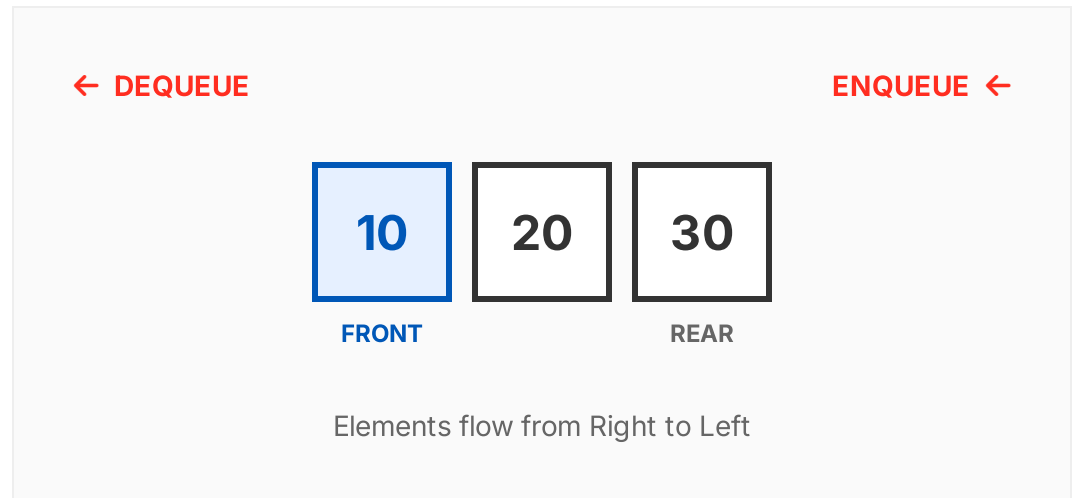
- **Function Call Stack:** Stores local variables & return addresses.
- **Expression Evaluation:** Converting Infix to Postfix notation.
- **Backtracking:** DFS graph traversal, maze solving.
- **Undo Operations:** Editors and browser history.

```
// Array-based stack implementation
class Stack {
    int arr[100];
    int top;
public:
    Stack() : top(-1) {}
    void push(int x) {
        if (top >= 99) return; // Overflow check
        arr[++top] = x;
    }
    int pop() {
        if (top < 0) return -1; // Underflow check
        return arr[top--];
    }
    int peek() { return arr[top]; }
    bool isEmpty() { return top == -1; }
};
```

QUEUES: FIFO PRINCIPLE

Queues process elements in arrival order, essential for fair scheduling and buffering.

- **FIFO (First-In, First-Out):** The first element added is the first one removed (like a line at a store).
- **Core Operations:**
 - **Enqueue:** Add to Rear ($O(1)$)
 - **Dequeue:** Remove from Front ($O(1)$)
- **Implementation:** Circular array or Linked List.
- **Applications:** Task scheduling, Breadth-First Search (BFS), Printer spooling.



```
#include <queue> std::queue<int> q; q.push(10);  
// Queue: [10] q.push(20); // Queue: [10, 20]  
q.push(30); // Queue: [10, 20, 30] cout <<  
q.front(); // Output: 10 q.pop(); // Queue:  
[20, 30]
```

CIRCULAR QUEUE IMPLEMENTATION

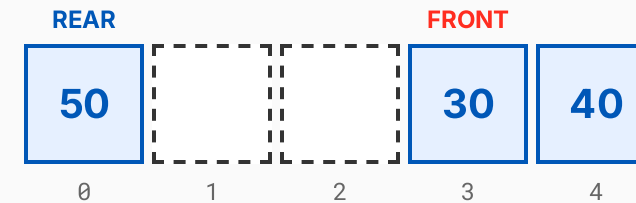
Circular queues eliminate wasted space by wrapping indices around array boundaries.

- **Problem:** Standard array queues drift to the right, leaving unused space at the front after dequeues.
- **Solution:** Treat the array as a circle. When the rear reaches the end, it wraps back to index 0.
- **Modulo Arithmetic:** The core mechanism for wrapping indices.
- **Efficiency:** Reuses freed space without shifting elements ($O(1)$ operations).

NEXT INDEX FORMULA

$\text{index} = (\text{index} + 1) \% \text{capacity}$

Visualizing Wrap-Around (Cap: 5)



Queue contains: [30, 40, 50]. Rear wrapped to 0.

```
void enqueue(int x) { if ((rear + 1) % size == front) return; // Full arr[rear] = x; rear = (rear + 1) % size; // Wrap count++; } int dequeue() { int x = arr[front]; front = (front + 1) % size; // Wrap count--; return x; }
```

TREES: HIERARCHICAL DATA

Trees organize data hierarchically, enabling efficient search, insertion, and traversal.

ROOT

Topmost node with no parent. The entry point.

PARENT / CHILD

Direct hierarchical relationship connected by edges.

LEAF

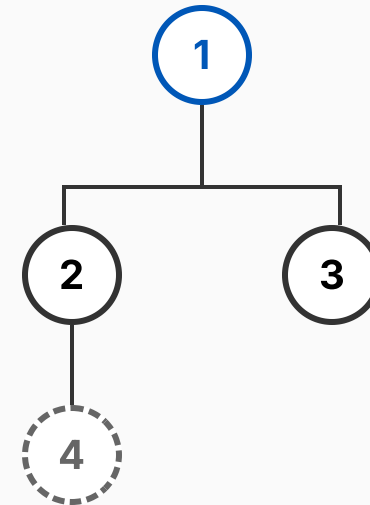
Node with no children (bottom of the tree).

HEIGHT

Length of the longest path from root to a leaf.

Common Applications:

File Systems, HTML DOM, Databases, AI Decision Trees



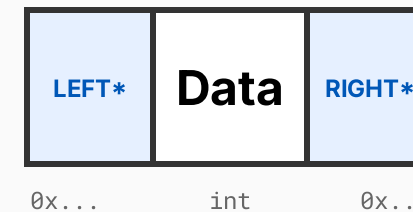
```
struct TreeNode {  
    int data;  
    std::vector<TreeNode*> children;  
};  
  
TreeNode* root = new TreeNode{1, {}};  
root->children.push_back(new TreeNode{2, {}});
```

BINARY TREES

Binary trees limit each node to two children, enabling efficient algorithms.

- **Two-Child Constraint:** Each node has at most two children, distinguished as *Left* and *Right*.
- **Special Types:**
 - **Full Binary Tree:** Every node has 0 or 2 children.
 - **Complete Binary Tree:** All levels filled (except possibly last, left-to-right).
- **Mathematical Property:** A binary tree of height h has at most $2^{h+1} - 1$ nodes.
- **Foundation:** Basis for BSTs, Heaps, and Expression Trees.

Node Memory Layout



```
struct BinaryTreeNode { int data;  
    BinaryTreeNode* left; BinaryTreeNode* right; };  
// Creating a simple root BinaryTreeNode* root  
= new BinaryTreeNode{1, nullptr, nullptr};  
root->left = new BinaryTreeNode{2, nullptr,  
    nullptr}; root->right = new BinaryTreeNode{3,  
    nullptr, nullptr};
```

BINARY TREE TRAVERSALS

Three traversal orders serve different algorithmic needs based on when the root is visited.

PREORDER

ROOT → Left → Right

Use Case: Creating a copy of the tree; Prefix expression evaluation.

```
void preorder(Node* n) { if (!n) return;
cout << n->data; // Visit preorder(n-
>left); preorder(n->right); }
```

INORDER

Left → **ROOT** → Right

Use Case: Getting sorted sequence from a Binary Search Tree (BST).

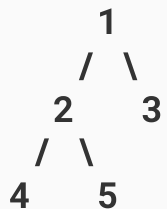
```
void inorder(Node* n) { if (!n) return;
inorder(n->left); cout << n->data; //
Visit inorder(n->right); }
```

POSTORDER

Left → Right → **ROOT**

Use Case: Deleting the tree (delete children before parent); Postfix evaluation.

```
void postorder(Node* n) { if (!n) return;
postorder(n->left); postorder(n->right);
cout << n->data; // Visit }
```



PREORDER OUTPUT

1 2 4 5 3

INORDER OUTPUT

4 2 5 1 3

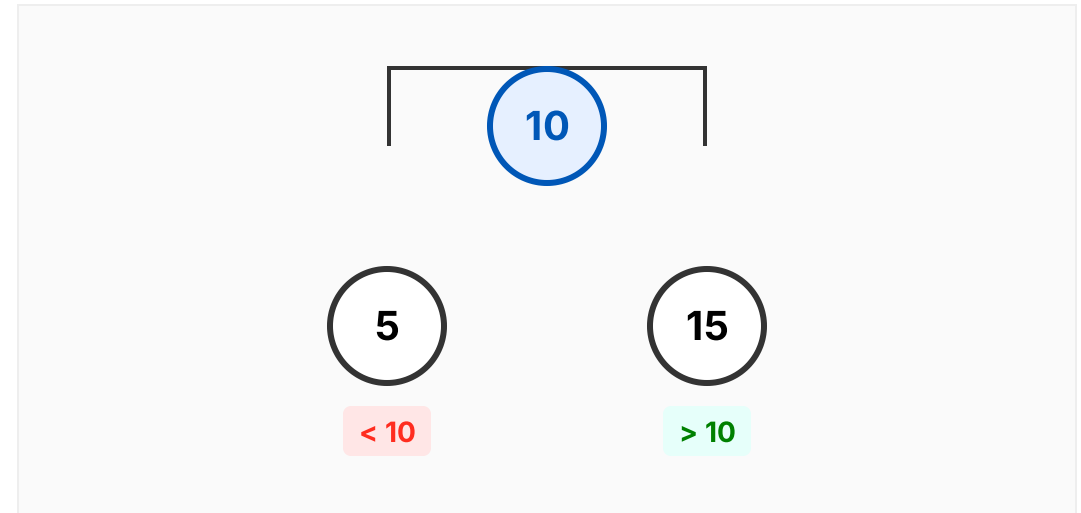
POSTORDER OUTPUT

4 5 2 3 1

BINARY SEARCH TREES (BST)

BST property ($\text{left} < \text{root} < \text{right}$) enables $O(\log n)$ search in balanced trees.

- **BST Property:** For any node, all keys in the left subtree are smaller, and all keys in the right subtree are larger.
- **Efficient Search:** Eliminates half the tree at each step (like Binary Search).
- **Inorder Traversal:** Visits nodes in strictly increasing sorted order.



AVERAGE CASE

$O(\log n)$

Balanced Tree

WORST CASE

$O(n)$

Skewed (Line)

```
Node* search(Node* root, int key) { if (root == nullptr || root->data == key) return root; if (key < root->data) return search(root->left, key); else return search(root->right, key); }
```


BST OPERATIONS

Insertion finds the correct leaf position; Deletion handles three structural cases.

INSERTION

Logic: Traverse down (Left if smaller, Right if larger) until a nullptr is found. Insert the new node there.

```
Node* insert(Node* root, int key) { if (root ==  
nullptr) return new Node{key, nullptr, nullptr};  
if (key < root->data) root->left = insert(root->  
left, key); else root->right = insert(root->  
right, key); return root; }
```

DELETION CASES

CASE 1: LEAF NODE

The simplest case. Just remove the node and update the parent's pointer to nullptr.

CASE 2: ONE CHILD

Bypass the node. Link the parent directly to the node's **only child**, then delete the node.

CASE 3: TWO CHILDREN

Cannot delete directly. Must maintain BST order.

Strategy: Find the **Inorder Successor** (Smallest node in the Right Subtree). Replace the target node's value with the Successor's value, then delete the Successor (which falls into Case 1 or 2).

BALANCED TREES

Self-balancing trees prevent $O(n)$ degeneration, ensuring $O(\log n)$ performance.

THE PROBLEM: DEGENERATION

Inserting sorted data into a standard BST creates a "skewed" tree, effectively becoming a Linked List. Search time degrades to $O(n)$.



THE SOLUTION: BALANCING

Algorithms that automatically adjust the tree structure (via rotations) to keep height logarithmic.

Mechanism: Tree Rotation

Rotations change the structure without violating the BST property. A "Right Rotation" lifts the left child up to become the new parent.

```
// Right Rotation (to fix Left-Left imbalance) Node*  
rightRotate(Node* y) { Node* x = y->left; Node* T2 =  
x->right; // Perform rotation x->right = y; y->left  
= T2; // Update heights (omitted) return x; // New  
root }
```

Note: Balancing guarantees that the height h is always $O(\log n)$, making all operations efficient.

TYPE	STRATEGY	USE CASE
AVL Tree	Strict (Height diff ≤ 1)	Read-heavy workloads

HEAPS: PRIORITY STRUCTURES

Heaps maintain parent-child ordering to enable $O(1)$ access to the maximum/minimum element.

MAX-HEAP

Parent nodes are always **greater than or equal to** their children. The Root is the maximum element.

MIN-HEAP

Parent nodes are always **less than or equal to** their children. The Root is the minimum element.

- **Complete Binary Tree:** Always filled from left to right. No gaps allowed.
- **Applications:** Priority Queues, Heap Sort, Graph Algorithms (Dijkstra, Prim).

ARRAY REPRESENTATION (IMPLICIT TREE)

90	80	70	60	50	40	30
0	1	2	3	4	5	6

LEFT CHILD

$$2*i + 1$$

RIGHT CHILD

$$2*i + 2$$

PARENT

$$(i-1) / 2$$

```
// Helper functions for 0-indexed heap
int parent(int i) { return (i-1)/2; }
int leftChild(int i) { return 2*i + 1; }
int rightChild(int i) { return 2*i + 2; }
// Array: [90, 80, 70, 60, 50, 40, 30] // Root: 90, Left: 80, Right: 70
```

HEAP OPERATIONS

Heapify maintains heap property through recursive swapping in $O(\log n)$ time.

INSERTION

Add element at the end (bottom-right), then **Bubble Up** by swapping with parent until heap property is restored.

Time: $O(\log n)$

EXTRACT MAX/MIN

Remove root. Move the last element to the root. Then **Bubble Down** (Heapify) by swapping with the larger/smaller child.

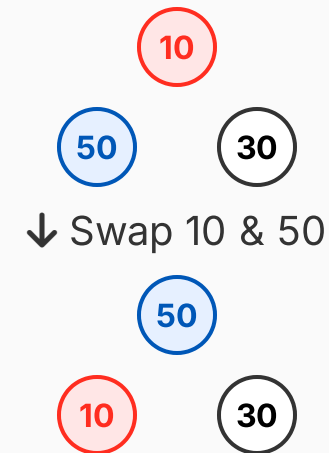
Time: $O(\log n)$

BUILD HEAP

Convert an unsorted array into a heap by calling Heapify on all non-leaf nodes.

Time: $O(n)$

VISUALIZING HEAPIFY DOWN (MAX HEAP)



```
void heapifyDown(int arr[], int n, int i) { int largest = i;
int left = 2*i + 1; int right = 2*i + 2; // Find largest
among root, left, right if(left < n && arr[left] >
arr[largest]) largest = left; if(right < n && arr[right] >
arr[largest]) largest = right; // Swap and recurse if root
isn't largest if(largest != i) { swap(arr[i], arr[largest]);
heapifyDown(arr, n, largest); } }
```

GRAPHS: MODELING NETWORKS

Graphs represent complex relationships through vertices and edges, enabling network analysis.

Vertex (V)

A node representing an entity (e.g., a User, a City, a Webpage).

Edge (E)

A connection between two vertices representing a relationship.

```
struct Graph { int V; // Number of vertices
vector<vector<int>> adjList; Graph(int
vertices) : V(vertices) { adjList.resize(V); }
void addEdge(int u, int v) {
adjList[u].push_back(v); //
adjList[v].push_back(u); // If undirected } };
```



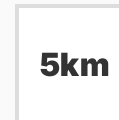
DIRECTED GRAPH

One-way connections (e.g., Twitter Follows, Web Links).



UNDIRECTED GRAPH

Two-way connections (e.g., Facebook Friends, Roads).



WEIGHTED GRAPH

Edges have values (e.g., Distance, Cost, Bandwidth).

KEY APPLICATIONS

Social Networks

GPS Navigation

Network Routing

GRAPH REPRESENTATIONS

Adjacency matrices offer $O(1)$ edge lookup while lists provide space efficiency for sparse graphs.

ADJACENCY MATRIX

SPACE
 $O(V^2)$

EDGE LOOKUP
 $O(1)$

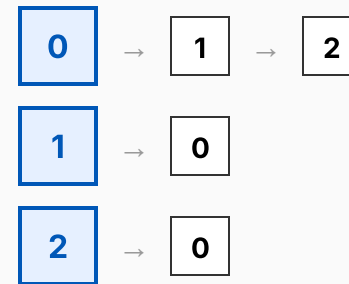
	0	1	2
0	0	1	1
1	1	0	0
2	1	0	0

```
// 2D Array (Dense Graphs) int adj[3][3] = { {0, 1, 1}, {1, 0, 0}, {1, 0, 0} }; // Check edge 0->1: adj[0][1] == 1
```

ADJACENCY LIST

SPACE
 $O(V + E)$

EDGE LOOKUP
 $O(\text{degree})$



```
// Array of Vectors (Sparse Graphs) vector<int> adj[3];  
adj[0] = {1, 2}; adj[1] = {0}; adj[2] = {0}; // Check edge:  
iterate adj[u]
```

DEPTH-FIRST SEARCH (DFS)

DFS explores as deep as possible along each branch before backtracking.

STRATEGY: GO DEEP

Uses a **Stack** (often the system call stack via recursion) to remember where to backtrack to.

ALGORITHM LOGIC

- 1 **Mark** the current node as visited.
- 2 **Explore** an unvisited neighbor recursively.
- 3 **Backtrack** if no unvisited neighbors remain.

```
void DFS(int u, vector<bool>& visited,
vector<int> adj[]) { visited[u] = true; cout <<
u << " "; // Process node // Visit all adjacent
vertices for (int v : adj[u]) { if
(!visited[v]) { DFS(v, visited, adj); } } }
```

KEY APPLICATIONS

Maze Solving

Cycle Detection

Topological Sort

Path Finding

Time Complexity: $O(V + E)$

BREADTH-FIRST SEARCH (BFS)

BFS explores neighbors layer by layer using a Queue, ideal for finding shortest paths.

THE STRATEGY

Like a ripple in a pond. Start at the source, visit all immediate neighbors (Distance 1), then all neighbors of neighbors (Distance 2), and so on.

Source

→

Dist 1

→

Dist 2

ALGORITHM STEPS

- 1 Push **Start Node** to Queue and mark visited.
- 2 While Queue is not empty:
- 3 **Pop** front node u .
- 4 For every neighbor v of u : If not visited, mark visited and **Push** to Queue.

Time Complexity: $O(V + E)$

```
void BFS(int start) { queue<int> q; q.push(start);
visited[start] = true; while (!q.empty()) { int u =
q.front(); q.pop(); cout << u << " "; for (int v : adj[u])
{ if (!visited[v]) { visited[v] = true; q.push(v); } } }
```

KEY APPLICATION: SHORTEST PATH

In an **Unweighted Graph**, BFS guarantees finding the shortest path (minimum number of edges) from the source to any other node.

HASH TABLES

Mapping keys to values for constant-time $O(1)$ average access.

HASH FUNCTION

Converts a Key (string, int) into an Index. Ideally distributes keys uniformly across the array.

$\text{Index} = \text{hash}(\text{Key}) \% \text{Size}$

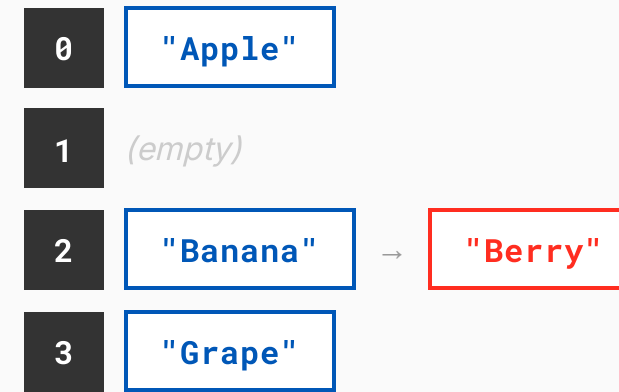
COLLISION HANDLING

When two keys map to the same index.

Chaining: Store a Linked List at each index.

Open Addressing: Find the next empty slot.

VISUAL: SEPARATE CHAINING



OPERATION	AVERAGE	WORST CASE
Insert / Search	$O(1)$	$O(n)$

```
// C++ STL Usage #include <unordered_map>
unordered_map<string, int> scores; // Insert  $O(1)$ 
scores["Alice"] = 95; scores["Bob"] = 88; // Search  $O(1)$  if
(scores.find("Alice") != scores.end()) { cout <<
scores["Alice"]; }
```

CHOOSING THE RIGHT DATA STRUCTURE

Correct selection transforms $O(n)$ bottlenecks into $O(1)$ or $O(\log n)$ efficiency.

Need fast access by index?

ARRAY / VECTOR

Need fast lookup by key (ID, Name)?

HASH TABLE

Need data always sorted?

BST / AVL

Processing tasks in order (FIFO)?

QUEUE

Processing nested operations (LIFO)?

STACK

Accessing highest priority item?

HEAP

Modeling networks or connections?

GRAPH

KEY TRADE-OFFS

Array vs. Linked List

Arrays offer **Cache Locality** and fast random access. Lists offer fast **Insert/Delete** at ends but suffer from pointer overhead.

Hash Table vs. Tree

Hash Tables are **$O(1)$** but unordered. Trees are **$O(\log n)$** but maintain order for range queries.

Rule of Thumb: Start with the simplest structure (Array/Vector). Optimize only when profiling identifies a bottleneck.

SUMMARY & KEY TAKEAWAYS

Data structures organize information to optimize specific operations like access, search, insertion, and deletion.

LINEAR

Arrays

Contiguous memory. Fast random access $O(1)$. Fixed size.

Linked Lists

Dynamic size. Fast insertion/deletion $O(1)$. Sequential access.

Stacks & Queues

Restricted access disciplines (LIFO / FIFO). Essential for algorithm control flow.

HIERARCHICAL

Binary Trees

Hierarchical data representation. Base for more complex structures.

BST & AVL

Sorted data. Efficient search, insert, delete $O(\log n)$.

Heaps

Priority-based access. Instant access to Max/Min $O(1)$.

COMPLEX

Hash Tables

Key-Value mapping. The gold standard for lookup speed $O(1)$.

Graphs

Modeling arbitrary relationships and networks. Traversed via BFS/DFS.

***Final Thought:** There is no "perfect" data structure. The right choice depends entirely on the specific requirements of your algorithm (e.g., Read-heavy vs. Write-heavy).*

FURTHER LEARNING

Mastery comes from practice. Combine theoretical study with hands-on coding.

RECOMMENDED BOOKS

→ **Introduction to Algorithms (CLRS)**

The comprehensive "Bible" of algorithms.

→ **Cracking the Coding Interview**

Practical application for job interviews.

PRACTICE PLATFORMS

→ LeetCode / HackerRank

→ CodeSignal

→ GeeksforGeeks

WHAT'S NEXT?

1. Algorithms Course

Sorting

Searching

Dynamic Programming

Greedy

2. Advanced Data Structures

Tries

Segment Trees

Disjoint Sets

B-Trees

3. System Design

Caching

Database Indexing

Load Balancing